

Prova del gg-Mmm-yyyy (90 minuti)

Cognome _____ **Nome:** _____

[illegible][illegible]

4) Utilizzando NumPy e OpenCV, implementare in Python la funzione *esercizio(img)* che riceve un'immagine a colori *img* (con tre byte per pixel) e deve eseguire le seguenti operazioni:

- 1) Convertire *img* in grayscale, memorizzando il risultato in un'immagine chiamata *img_g*.
- 2) Se la larghezza di *img_g* è maggiore della sua altezza, ruotarla di 90 gradi in senso antiorario (equivale a scambiare le righe con le colonne).
- 3) Binarizzare l'immagine utilizzando l'algoritmo di Otsu.
- 4) Applicare al risultato del passo precedente un'operazione morfologica di dilatazione con un elemento strutturante di forma quadrata e lato di 9 pixel.
- 5) Trovare il più esteso gruppo di pixel di foreground contigui nell'immagine ottenuta al passo precedente: eliminare tutti gli altri pixel di foreground.
- 6) Restituire una copia di *img_g* in cui è dimezzata la luminosità dei pixel le cui posizioni sono state individuate al passo precedente.

Promemoria: alcune funzioni che potrebbero essere utili

```
morphologyEx(src, op, kernel[, dst[, anchor[, iterations[, borderType[, borderValue]]]]) -> dst
cvtColor(src, code[, dst[, dstCn]]) -> dst
GaussianBlur(src, ksize, sigmaX[, dst[, sigmaY[, borderType]]]) -> dst
threshold(src, thresh, maxval, type[, dst]) -> retval, dst
connectedComponentsWithStats(image) -> retval, labels, stats, centroids
getStructuringElement(shape, ksize[, anchor]) -> retval
```

Promemoria: alcune costanti che potrebbero essere utili

```
cv.CC_STAT_WIDTH, cv.CC_STAT_HEIGHT, cv.CC_STAT_AREA
cv.COLOR_BGR2GRAY, cv.COLOR_GRAY2BGR, cv.COLOR_BGR2RGB, cv.THRESH_BINARY, cv.THRESH_OTSU
cv.MORPH_ELLIPSE, cv.MORPH_DILATE, cv.MORPH_RECT, cv.MORPH_ELLIPSE
```